

AI y BigQuery en Google Cloud

Universidad Austral

Facultad de Ingeniería

Primer Cuatrimestre 2026

Profesores

Mora Villa Abrille (BVilla@austral.edu.ar)

Rodrigo Pazos (RPazos@austral.edu.ar)

BigQuery — el data warehouse de GCP

Data warehouse **serverless**: SQL sobre petabytes, pay-as-you-go, sin infraestructura.

Cargar datos: 3 caminos con trade-offs distintos

Método	Automatizable	Cuándo elegirlo
Console (CSV upload)	No	Exploración, datasets chicos
CLI <code>bq load</code> desde GCS	Sí (scripteable)	Pipelines batch
DDL <code>CREATE TABLE AS</code>	Sí (SQL)	Derivar tablas ya en BQ

BigQuery ML — ML sin mover los datos

Ejemplo con el dataset de taxis de NYC:

```
-- Entrenar: predecir tarifa en base a distancia, pasajeros y hora
CREATE OR REPLACE MODEL nyctaxi.fare_model
OPTIONS(model_type='linear_reg', input_label_cols=['fare_amount']) AS
SELECT
  fare_amount,
  trip_distance,
  passenger_count,
  EXTRACT(HOUR FROM pickup_datetime) AS pickup_hour
FROM nyctaxi.2018trips
WHERE fare_amount > 0 AND trip_distance > 0;
```

```
-- Predecir: ¿cuánto costaría un viaje de 5km con 2 pasajeros a las 18h?
SELECT * FROM ML.PREDICT(MODEL nyctaxi.fare_model,
  (SELECT 5.0 AS trip_distance, 2 AS passenger_count, 18 AS pickup_hour));
```

Trade-off: menos modelos que Vertex AI, pero datos **nunca salen de BQ** → cero infra de ML.

Los modelos se pueden [exportar](#) a Cloud Storage (TensorFlow, XGBoost, ONNX) para usarlos fuera de BQ.

El mapa de AI en GCP

Agent Platform (ex Vertex AI) es la plataforma unificada de ML/AI de GCP — renombrada en 2026. Engloba: **Model Garden**, Agent Builder, RAG Engine, Notebooks, y la Gemini API.

Agent Studio (ex Vertex AI Studio) es la interfaz visual dentro de Agent Platform para prototipar con modelos generativos.

Tres niveles de abstracción — de más fácil a más control:

	APIs pre-entrenadas	Modelos generativos	Modelo propio
Servicios	Vision AI, Speech-to-Text, Document AI, Translation	Gemini (Flash/Pro), Imagen, Chirp, Llama, Mistral	Vertex AI Training, AutoML, BigQuery ML
Cómo se usa	Llamada API, sin config	Prompt + configuración	Dataset + entrenamiento
Tiempo de setup	Minutos	Minutos	Horas / días
Costo típico	~\$1-5 / 1000 requests	~\$0.05-5 / 1M tokens	~\$50-500+ / run

Principio de ingeniería: empezar siempre por la izquierda.

Decisión 1: ¿API, generativo, o modelo propio?

Escenario A: "Extraer texto de 50,000 facturas"

-  **API pre-entrenada (Document AI)** — hecho para esto, rápido, barato
- ~~Gemini~~ — 10x más caro y más lento para esta tarea

Escenario B: "Chatbot que responda sobre nuestros productos"

- ~~API pre-entrenada~~ — no entiende contexto
-  **Gemini + Grounding** — le das tu documentación, responde con citas

Escenario C: "Detectar fraude en transacciones"

- ~~Gemini~~ — no sirve para clasificación numérica a escala
-  **Modelo propio (AutoML / BQML)** — entrenar con tus datos

Señal	Elegir
Tarea estándar y bien definida (OCR, traducción, STT)	API pre-entrenada
Necesita razonamiento, lenguaje natural, creatividad	Modelo generativo
Datos tabulares, predicción numérica, escala masiva	Modelo propio

Model Garden — elegir el modelo correcto

Catálogo de **200+ modelos**, todos desplegados desde un solo lugar.

Gemini: Flash vs Pro

Aspecto	Flash	Pro
Latencia	~0.5s	~2-5s
Costo	~\$0.05 / 1M tokens	~\$1.25 / 1M tokens (25x)
Razonamiento	Bueno	Significativamente mejor

Flash para el **80% de los casos**. Pro cuando la calidad justifica el costo.

¿Google o Open Source?

- **Google (Gemini):** managed, zero-ops, multimodal nativo, SLA enterprise
- **Open Source (Llama, Mistral):** control total, on-premise, sin lock-in, menor costo

Decisión 2: ¿Cómo le doy conocimiento al modelo?

El modelo base **no sabe nada de tu empresa**. Tres caminos:

Grounding (el más simple)

Conectar el modelo a fuentes externas en tiempo de inferencia.

- **Google Search**: datos públicos actualizados
- **Tus datos**: documentos propios (manuales, políticas)
- Setup: **minutos**. El modelo **cita fuentes**.

RAG — Retrieval Augmented Generation (el más usado)

Tus docs → chunks → embeddings → Vector Search (índice)
Pregunta → embedding → chunks similares → LLM responde con contexto

Setup: **horas**. Más control sobre qué y cómo se recupera.

Fine-tuning (el más costoso)

Reentrenar el modelo con tus datos (pares input/output).

Setup: **días**. Cambia el **comportamiento** del modelo, no su conocimiento.

Ejemplo: Grounding con Google Search

```
from google import genai
from google.genai.types import GenerateContentConfig, Tool, GoogleSearch

client = genai.Client(vertexai=True, project="my-project", location="global")
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="¿Cuál es la cotización del dólar en Argentina hoy?",
    config=GenerateContentConfig(
        tools=[Tool(google_search=GoogleSearch())]
    ),
)
print(response.text)
# response.candidates[0].grounding_metadata → fuentes usadas
```

Sin **Grounding**: el modelo **inventa** un número. Con Grounding: busca en Google, responde con datos reales y **cita las fuentes**.

Decisión 3: ¿Chatbot o Agente?

Chatbot: genera texto

Usuario: "¿Cuánto cuesta el envío a Córdoba?"
Modelo: "Según nuestra documentación, cuesta \$5,000."

Agente: ejecuta acciones

Usuario: "Reservame un envío a Córdoba para mañana"
Modelo → llama: check_availability(dest="Córdoba")
Sistema → {available: true, price: 5000}
Modelo → llama: create_booking(shipment_id="SHP-123")
Sistema → {confirmed: true, tracking: "TRK-456"}
Modelo: "Listo, reservé el envío. Tu tracking es TRK-456."

El modelo no accede a tus sistemas — genera un **pedido estructurado** (ej: `book_flight("BA", "CBA", "viernes")`). Tu aplicación recibe ese pedido, lo ejecuta, y le devuelve el resultado.

Ejemplo: Function Calling con Gemini

```
from google import genai
from google.genai.types import GenerateContentConfig

client = genai.Client(vertexai=True, project="my-project", location="us-central1")

# 1. Definís las funciones que el modelo puede usar
def get_weather(location: str) -> dict:
    """Devuelve el clima actual de una ubicación."""
    return requests.get(f"https://api.weather.com/{location}").json()

def book_flight(origin: str, destination: str, date: str) -> dict:
    """Reserva un vuelo entre dos ciudades."""
    return requests.post("https://api.flights.com/book",
        json={"from": origin, "to": destination, "date": date}).json()

# 2. Le pasás las funciones como tools – Gemini decide cuál usar
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="Necesito volar de Buenos Aires a Córdoba el viernes. ¿Cómo va a estar el clima allá?",
    config=GenerateContentConfig(tools=[get_weather, book_flight]),
)
# Gemini llama a get_weather("Córdoba") Y a book_flight("Buenos Aires", "Córdoba", "2026-06-06")
```

¿Cómo sabe Gemini qué hace cada función? Lee el **nombre**, los **type hints**, y el **docstring**:

`get_weather(location: str) -> dict` + `"""Devuelve el clima actual..."""` → Gemini entiende cuándo usarla y qué parámetros pasarle. El SDK convierte esto a un JSON schema automáticamente.

Prompt Engineering — la herramienta más barata

Antes de cambiar modelo, agregar RAG, o fine-tunear: **mejorar el prompt es gratis.**

Técnica	Ejemplo concreto
System Instructions	"Sos un analista financiero. Respondé solo con datos, nunca especules."
Output Structuring	"Respondé en JSON con keys: risk_level, factors, recommendation"
Few-shot	Dar 2 ejemplos de input→output antes del input real
Chain of Thought	"Antes de responder, razoná paso a paso"

Temperature — cómo funciona

El modelo predice el **siguiente token** calculando una probabilidad para cada palabra posible. Temperature escala esas probabilidades:

- **Temp 0**: siempre elige el token más probable → determinístico
- **Temp < 1**: los tokens probables se vuelven aún más probables → más enfocado
- **Temp > 1**: tokens menos probables ganan chances → más diverso pero más errático

Ejemplo — completar "Buenos Aires es la capital de ____":

Temperature	Resultado	Distribución
0	"Argentina" siempre	100% token más probable
0.5	"Argentina" casi siempre	95% / 4% / 1%
1.5	Respuestas variadas e inesperadas	60% / 15% / 10% / 15%

Extracción de datos → **0.0-0.2**. Q&A → **0.3-0.5**. Creatividad → **0.8-1.5**.

Responsible AI — restricciones reales

No es solo ética — son **restricciones de diseño** que afectan tu arquitectura.

Lo que GCP garantiza

- **Datos del cliente NO se usan** para entrenar modelos de Google
- **Safety Filters** activos por default — bloquean contenido dañino
- **SynthID** — watermark invisible en contenido generado por IA

Impacto en diseño de sistemas

Restricción	Impacto
Safety filters rechazan inputs legítimos	Manejar rehúso del modelo
Datos sensibles no salen del proyecto	Respetar data residency
Alucinaciones siempre posibles	Grounding/RAG + mostrar fuentes
No hay determinismo garantizado	Misma pregunta ≠ misma respuesta
Rate limits y cuotas	Retries, caching, fallback

Mapa de decisiones — resumen

Pregunta	Respuesta	Solución
¿Qué tipo de tarea es?	Estándar (OCR, traducción, STT)	API pre-entrenada
	Razonamiento / lenguaje natural	Modelo generativo (Gemini)
	Predicción numérica / datos tabulares	BigQuery ML o Vertex AI Training
¿Necesita datos de mi empresa?	Sí, datos que cambian	Grounding o RAG
	Sí, cambiar comportamiento del modelo	Fine-tuning
¿Necesita ejecutar acciones?	Sí	Agent + Function Calling
	No	Chatbot
¿Necesita ser barato en prod?	Sí	Distillation (Pro → Flash)